IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Mitigating the Higher-Density Bias of Diffusion Models under Recursive Training on Synthetic Data

Team Name: Tensor Titans *Team Members: 24B0924*

Abstract

When a generative model is trained on images made by another generative model, its mistakes pile up. We study this with a small unconditional diffusion model on an imbalanced subset of CIFAR-10: five common classes with about 4,900 images each and five rare classes with 1,500 down to 50 images (100:1). We run the loop D0→M0→D1→M1→D2→M2→D3, training every model from scratch on the images generated by the previous one. Already M0 over-produces the common classes (94.8% of its recognised images, against 90.2% for a perfect generator), and recursion makes the class mix drift further: the KL divergence to the original mix grows from 0.042 to 0.284 in two generations. To reduce this we weight the diffusion loss of each training image by $p_{\text{orig}}(c)/p_{\text{gen}}(c)$ for its class c (CDR-W). With the same images and compute, CDR-W removes 77% of the baseline's extra KL divergence and improves tail retention, but it does not stop the loss of image quality, and it does not help the rarest class. A training-free sampling step that finishes the second-generation images with M0 lowers the share of unrecognisable images from 36% to 22%. A test interface runs all models live on held-out test images.

1 Introduction

Generative models are now often trained on data that already contains generated images. If this happens repeatedly, each model inherits the errors of the one before it, and the rare parts of the data tend to vanish first [1]. This matters beyond image quality: rare classes and under-represented groups are the first to be lost [3]. Diffusion models are especially prone to it, since their denoiser pulls samples towards the dense, common regions of the data [4].

The project asks us to build a controlled density-imbalanced dataset, train an unconditional diffusion model on it, retrain on generated data for up to two rounds, measure the bias towards the high-density classes, and propose a mechanism that reduces this bias. Deep learning is used for the generator (a DDPM [7]), for the classifiers that measure class composition, and for the DINOv2 features [11] used in the quality metrics. The mitigation changes the training loss of the network itself.

Section 2 gives the workflow and Section 3 the related work. Our methods, before and after the prep-presentation review, are in Section 4, the dataset in Section 5, the experiments in Section 6 and the results in Section 7. Section 8 describes the test interface, Section 9 is the plan for the novelty assessment and Section 10 concludes.

2 Project Workflow

- Read the brief and the papers on recursive training; chose CIFAR-10 at 32×32 so everything fits on Kaggle (2×T4, 12-hour sessions).
- Built the imbalanced split D0: five head classes of 4,800–5,000 images, five tail classes of 1,500/800/400/150/50 images.
- Built a resumable pipeline for the loop, keeping every dataset and model.

- Designed the measurement (two classifiers) and the bias, quality and diversity metrics.
- Before the review: tried mitigation by selecting images, then by weighting each image with a nearest-neighbour ratio. Neither worked well enough (Section 4.1).
- After the review: switched to training every model from scratch, made the measurement more reliable, and built the class-weighted loss (CDR-W).
- Ran the final experiment (9 h 26 min on 2×T4), evaluated it, and built a Gradio test interface.

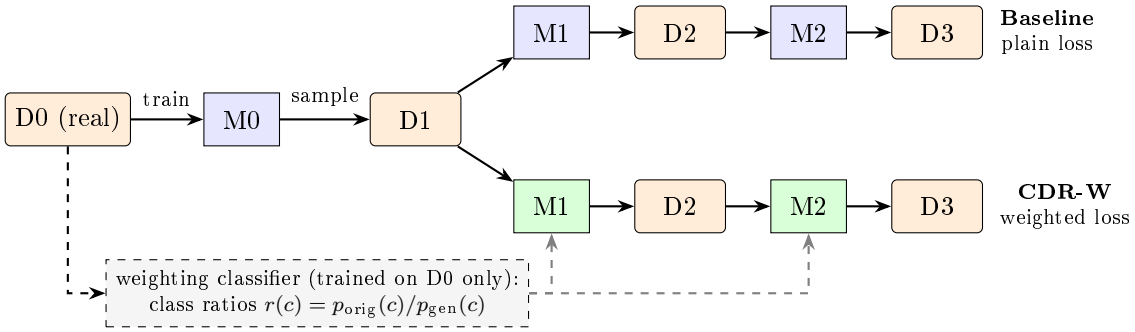


Figure 1: The recursive loop and the two chains. Each model is trained from scratch on the 27,400 images made by the previous one. Both chains share M0 and D1; CDR-W only changes the loss weights.

3 Literature Survey

Recursive training. Shumailov et al. [1] show that models trained on their predecessors' output lose the tails of the original distribution first, and later collapse to a narrow distribution. Alemohammad et al. [2] study this for image generators and show that a fully synthetic loop loses quality or diversity within a few generations unless fresh real data is added. Our baseline is this fully synthetic loop. Wyllie et al. [3] describe the same effect as a fairness feedback loop in which the majority group grows at the expense of minorities.

Diffusion models and density. Um et al. [4] explain that the denoiser of a diffusion model is biased towards the majority of the data, and propose minority guidance at sampling time. Qin et al. [5] show that on long-tailed CIFAR the tail classes lose diversity and drift towards head classes, and fix this with a regulariser for class-conditional models. Our model is unconditional, so we cannot use their method directly, but it pushed us towards changing the training objective.

Label shift. When only the class proportions change between two datasets, the standard correction is to weight each sample by $p_{\text{target}}(c)/p_{\text{train}}(c)$. Lipton et al. [6] estimate these ratios with a classifier. Generated data differs from the original mostly in class proportions, so we apply the same idea to the diffusion loss.

4 Proposed Approach or Approaches

4.1 Work done before prep-presentation review

We built the pipeline on Kaggle: M0 is a DDPM trained for 20,000 steps on D0, and D1 is sampled with DDIM. In this first version M1 and M2 were fine-tuned from the previous model (8,000 steps)

to save GPU time.

Run A: selecting images. We compared the baseline with three chains: KDR selection (generate twice as many images and keep half, chosen with a nearest-neighbour density ratio), an oracle selection using the classifier, and a chain that mixed 50% real images into each round. KDR selection lowered the measured drift (bias mitigation rate 0.57 on tail retention), but it works by throwing images away, so it does not make the *model* less biased. The first split also had a problem: it put horse among the common classes and truck among the rare ones, so degraded images of common classes were counted as rare ones and the baseline's tail seemed to *grow* (tail retention 1.21 to 2.19). We changed the split and dropped selection.

Run B: nearest-neighbour loss weights (KDR-W). With the corrected split we moved the mitigation into the loss. Each image was weighted by a nearest-neighbour estimate of $p_{\text{real}}(x)/p_{\text{gen}}(x)$ in DINOv2 space. It lowered KL (0.124 against 0.264) and FD (612 against 865), but the weights favoured common classes: the common-class share in its weighted training data was 95.2%, higher than in its pool (93.1%). Real and generated images differ mostly in quality, so this ratio rewards realistic-looking images, and those are mostly common-class images. Two other problems showed up. About 37% of the final images could not be recognised by the classifier, and the baseline's tail again seemed to grow. Also, a fine-tuned model keeps most of its parent's weights, so it cannot really accumulate the bias. This led to the changes below.

4.2 Work done after prep-presentation review

Recursion from scratch. M0, M1 and M2 are all trained from random initialisation with one recipe (20,000 steps, batch 256). Each generation is trained on exactly the 27,400 images made by the previous model, with no filtering and no real images.

Diffusion model. An unconditional UNet from `diffusers` [13] with 24.3M parameters (about two thirds of the usual DDPM CIFAR-10 model, so it counts as "slightly small"): channel widths 64/128/256/256, two residual blocks per level, self-attention at 16×16, dropout 0.1. It is trained to predict the added noise,

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0,\, t,\, \epsilon}\big\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t)\big\|^2, \tag{1}$$

with a linear noise schedule ($T = 1000$) and an exponential moving average of the weights. Images are generated with deterministic DDIM [8] in 50 steps.

CDR-W: class-density-ratio weighted training. Because the model is unconditional, its loss is an average over images, so each class is learned in proportion to how often it appears in the training set. If the previous model produced too many airplanes, the next model learns even more of them. CDR-W multiplies the loss of each image by a weight that depends on its class $\hat{c}$:

$$\mathcal{L}_{\text{CDR-W}} = \frac{1}{|B|}\sum_{i \in B} w_i \left\|\epsilon_i - \epsilon_\theta(x_{t,i}, t_i)\right\|^2, \qquad w_i = \text{clip}\Big(\frac{\tilde{p}_0(\hat{c}_i)}{\hat{p}_g(\hat{c}_i)},\, 0.1,\, 10\Big), \tag{2}$$

where $\hat{p}_g$ is the class mix of the generated training set and $\tilde{p}_0$ is the original class mix. In expectation this is the same as training on data with the original class proportions. We estimate the classes with a ResNet-18 trained only on D0 (class-balanced sampling, blur and noise augmentation, 91.3%

accuracy on held-out D0 images). Its target $\tilde{p}_0$ is what it measures on held-out real images, which removes its own per-class bias. Images it is not confident about (below 0.8) keep weight 1. Weights are normalised to mean 1; the effective sample size stayed above 90%. CDR-W uses the same images and compute as the baseline, keeps the generator unconditional, and never shows it a real image.

Before choosing this we tried two label-free ratios in DINOv2 space (a nearest-neighbour ratio and a ratio over k-means clusters) on a proxy of blurred, re-sampled real images. Both recovered only part of the tail, so we kept the class-level version. They remain in the code as ablations.

Measurement. We measure the class mix of every generated set in two ways: a ResNet-18 trained on the full balanced CIFAR-10 (blur and noise augmentation, 90.95% test accuracy; an image counts only if its confidence is at least 0.8), and the vote of the 10 nearest real CIFAR-10 images in DINOv2 ViT-S/14 space. Both are compared with a *perfect generator*, meaning the same measurement applied to held-out real test images in the original proportions, so the classifiers' own errors do not count as bias.

Metrics. The common group is airplane, automobile, ship, truck and frog; the mid group is bird and deer; the low group is cat, dog and horse. With p the measured class mix and $p^\star$ that of the perfect generator:

- **Tail retention ratio (TRR)** $= \sum_{\text{tail}} p \,/\, \sum_{\text{tail}} p^\star$. Ideal 1; below 1 means the tail is under-produced. This is the direct "are rare classes disappearing" number.
- **KL divergence** $\mathrm{KL}(p^\star \| p)$, ideal 0. It also picks up drift *inside* the common group, which TRR misses.
- β, the slope of $\log p(c)$ against $\log p^\star(c)$. $\beta > 1$ means big classes get bigger.
- **Tail coverage ratio (TCR)**: coverage [10] of the sparsest 25% of real images divided by that of the densest 25%. It needs no labels, so it does not depend on a classifier.
- **Quality and diversity**: Fréchet distance (FD) and recall in DINOv2 space, and the share of unrecognised images.
- **Bias mitigation rate (BMR)** $= 1 - |m_{\text{CDR-W}} - m^*| / |m_{\text{base}} - m^*|$ for a metric m with ideal value m^*: the share of the baseline's excess bias that CDR-W removes (1 = all, 0 = none).

5 Data set Details

We use CIFAR-10 [12]: 60,000 colour images of 32×32 pixels in 10 classes (50,000 for training, 10,000 for testing). The pipeline downloads it from the Hugging Face Hub (`uoft-cs/cifar10`) and checks the shapes and labels. From the training set we sampled the class sizes in Table 1 (seed 0). The head classes are almost equal and the tail decays to 50 images, giving a 100:1 imbalance. We put classes that are often confused in the same group (automobile/truck and airplane/ship in the head, cat/dog and deer/horse in the tail), so a misclassified image mostly stays in its group. The indices are saved in `run/data/split_indices.npy`.

Group	Class	CIFAR-10 id	Images in D0	Share
High (head)	airplane	0	5,000	18.25%
	automobile	1	4,900	17.88%
	ship	8	4,800	17.52%
	truck	9	4,850	17.70%
	frog	6	4,950	18.07%
Mid (tail)	bird	2	1,500	5.47%
	deer	4	800	2.92%
Low (tail)	cat	3	400	1.46%
	dog	5	150	0.55%
	horse	7	50	0.18%
Total			27,400	100%

Table 1: The imbalanced dataset D0 (common classes 89.4%, imbalance 100:1).

Pre-processing. For the diffusion models, images are scaled to $[-1, 1]$ and randomly flipped horizontally; there is no other augmentation, so the model learns the class proportions it is given. Each generated set D1, D2, D3 has 27,400 images, the same size as D0. The measurement classifier uses the full balanced training set with random crop, flip, blur and noise, and the 10,000 test images give the perfect-generator reference. The weighting classifier uses only D0 (15% held out). DINOv2 features are computed on images resized to 224×224.

6 Experiments

Our code is at https://github.com/kolanusarvajith/recursive-density-bias. It contains data preparation, diffusion training and sampling, the classifiers, the mitigation, the metrics and the evaluation, plus a Kaggle notebook and the test interface.

Hardware. Kaggle with 2×NVIDIA T4 (16 GB), PyTorch 2.11. M0 is trained on both GPUs (DistributedDataParallel); then the baseline and CDR-W chains run in parallel, one per GPU, using two-step gradient accumulation so the effective batch stays 256.

	Diffusion models (M0, M1, M2)	Classifiers (ResNet-18)
Optimiser	AdamW, no weight decay	SGD, momentum 0.9, w.d. 5×10^{-4}
Learning rate	2×10^{-4}, 1,000 warm-up steps, then constant	OneCycle, max 0.1
Batch, length	256, 20,000 steps	256, 20 epochs (30 for the weighter)
Other	gradient clip 1.0, fp16, EMA 0.9995	confidence threshold 0.8

Table 2: Training settings, identical for every generation and both chains.

Run time. M0 took 1 h 46 min; each of M1 and M2 took about 3 h (3 h 20 min for CDR-W); sampling 27,400 images took 10–20 min. The whole run took 9 h 26 min (about 19 GPU-hours).

All experiments, including the ones that failed.

1. A 2-D toy recursion with a small diffusion model. It over-produced the rarest component instead of losing it, so it did not reproduce the image case and we did not use it as evidence.

2. A proxy with blurred, re-sampled real CIFAR-10 images in DINOv2 space, to compare weighting methods. Label-free ratios recovered only part of the tail; class-level ratios worked best, which led to CDR-W.
3. Tiny CPU runs to test every stage (not results).
4. Run A and Run B (Section 4.1).
5. The final run: from-scratch recursion, baseline against CDR-W (Section 7).
6. Training-free improvements of the trained checkpoints on CPU: weight interpolation, classifier guidance, and refinement with M0 (Section 7.4).

7 Results

All numbers are for 27,400 generated images per set, and shares are over recognised images. D1 is the output of M0 and is shared by both chains. We ran one seed, so small differences may be noise.

Set	Model	High	Mid	Low	Unrecognised
Original data D0	–	89.4%	8.39%	2.19%	–
Perfect generator	–	90.2%	7.85%	1.98%	8.8%
D1	M0	94.8%	3.78%	1.40%	21.0%
D2	baseline M1	95.1%	3.16%	1.70%	27.6%
D3	baseline M2	94.9%	3.02%	2.13%	35.5%
D2	CDR-W M1	94.3%	3.85%	1.84%	28.6%
D3	CDR-W M2	93.4%	4.05%	2.59%	37.6%

Table 3: Share of the high, mid and low groups in each generated set.

Chain	Set	TRR	TRR (k-NN)	KL	β	TCR	Recall	FD
both	D1	0.526	0.462	0.0416	1.182	3.24	0.304	431.3
Baseline	D2	0.494	0.686	0.1213	1.240	8.34	0.151	754.7
Baseline	D3	0.523	0.955	0.2841	1.200	20.99	0.073	1072.4
CDR-W	D2	0.578	0.683	0.0625	1.192	8.09	0.125	708.9
CDR-W	D3	0.676	0.981	0.0662	1.191	15.49	0.059	993.7

Table 4: Bias, quality and diversity metrics (ideal: TRR 1, KL 0, β 1; lower FD and higher recall are better).

	airplane	auto.	ship	truck	frog	bird	deer	cat	dog	horse
Original D0	18.25	17.88	17.52	17.70	18.07	5.47	2.92	1.46	0.55	0.18
Perfect gen.	17.46	18.65	18.09	18.13	17.83	5.02	2.83	1.21	0.54	0.23
D1 (M0)	23.18	18.76	19.20	12.90	20.80	2.87	0.91	1.06	0.19	0.15
D2 baseline	30.61	16.73	16.05	8.51	23.23	2.65	0.51	1.46	0.14	0.09
D3 baseline	**41.38**	13.04	13.00	**4.34**	23.09	2.63	0.39	1.93	0.08	0.12
D2 CDR-W	25.91	18.03	17.34	11.49	21.55	3.19	0.66	1.54	0.20	0.11
D3 CDR-W	27.12	18.20	16.10	11.65	20.27	3.41	0.65	2.25	0.27	0.07

Table 5: Per-class share of the generated images (%).

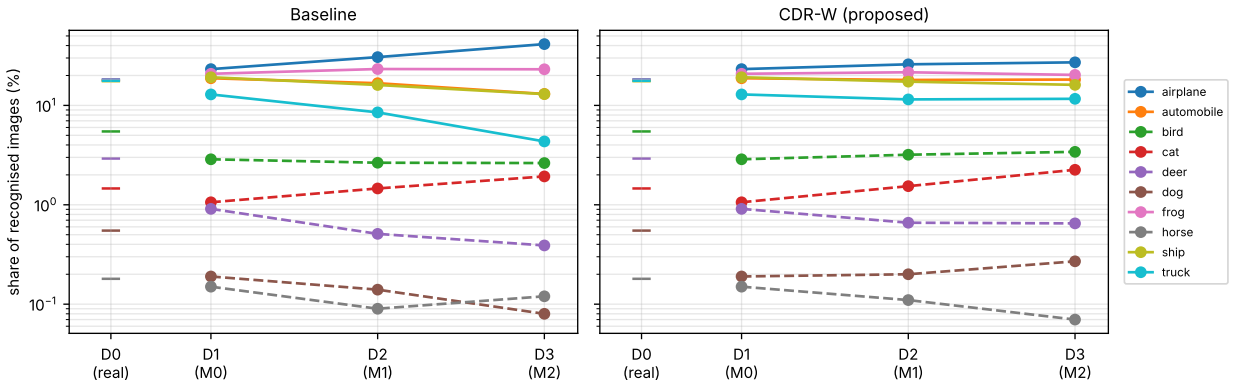


Figure 2: Per-class share across generations (log scale; solid = common classes, dashed = tail; markers on the left are the original shares).

7.1 Baseline: does recursion increase the bias?

- **M0 is already biased.** The common group takes 94.8% of its output against 90.2% for a perfect generator, and the tail is produced at about half its rate (TRR 0.53, or 0.46 with k-NN). Deer is hit hardest (0.9% against 2.9%).
- **The class mix drifts further.** KL grows from 0.042 to 0.121 to 0.284, and β stays above 1. Most of the change is *inside* the common group: airplane grows from 23% to 41% while truck falls from 13% to 4%. The five common classes have almost the same size in D0, so size alone does not explain it. Our guess is that the model reproduces some classes more easily and each generation amplifies that.
- **The tail group as a whole does not shrink.** Classifier TRR stays near 0.5 and k-NN TRR even rises. Deer and dog keep falling, but cat grows. So within two generations we see the first model's bias and the concentration on a few classes, but not a steady loss of the whole tail.
- **Quality collapses.** FD more than doubles (431 to 1072), recall falls from 0.30 to 0.07, and unrecognised images rise from 21% to 36%. The training loss also goes down each generation (0.0306, 0.0254, 0.0237), because the data gets easier to fit.

7.2 Does CDR-W reduce the bias?

Metric (D3)	Baseline	CDR-W	Ideal	BMR
KL to perfect generator	0.2841	0.0662	0	**0.77**
TRR (classifier)	0.523	0.676	1	0.32
TRR (DINOv2 k-NN)	0.955	0.981	1	0.58
Common-group share	94.9%	93.4%	90.2%	0.32
FD (lower is better)	1072.4	993.7	–	–
Recall (higher is better)	0.073	0.059	–	–
Unrecognised	35.5%	37.6%	8.8%	–

Table 6: Mitigation at the final generation. BMR is the share of the baseline's deviation from the ideal that CDR-W removes.

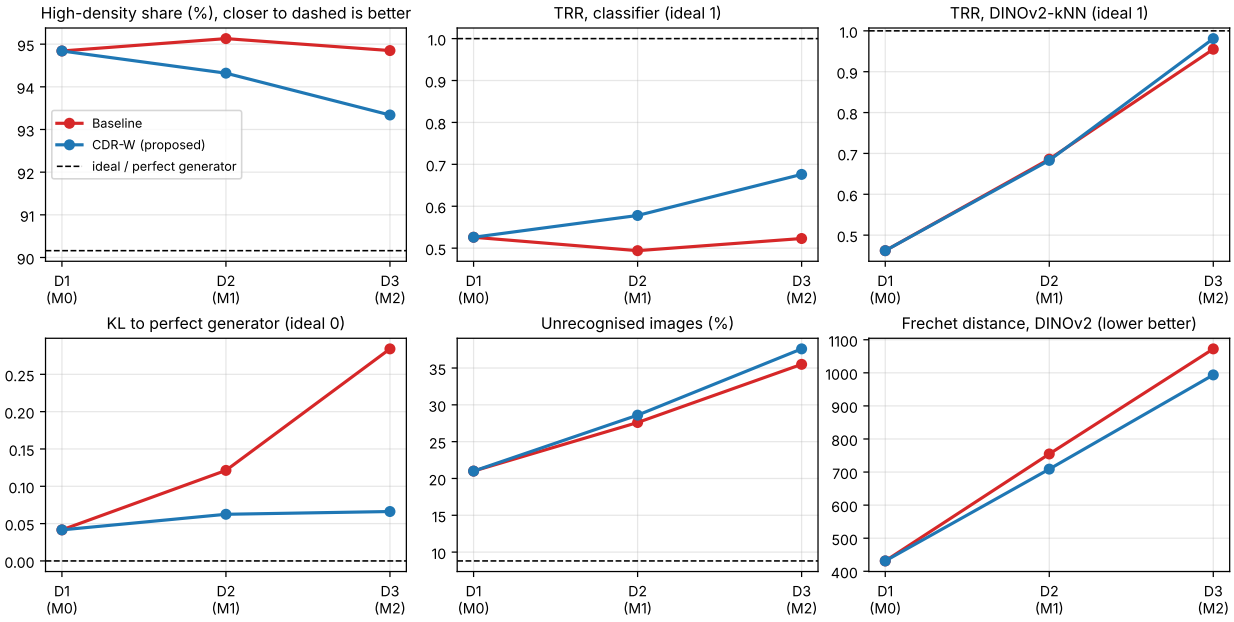
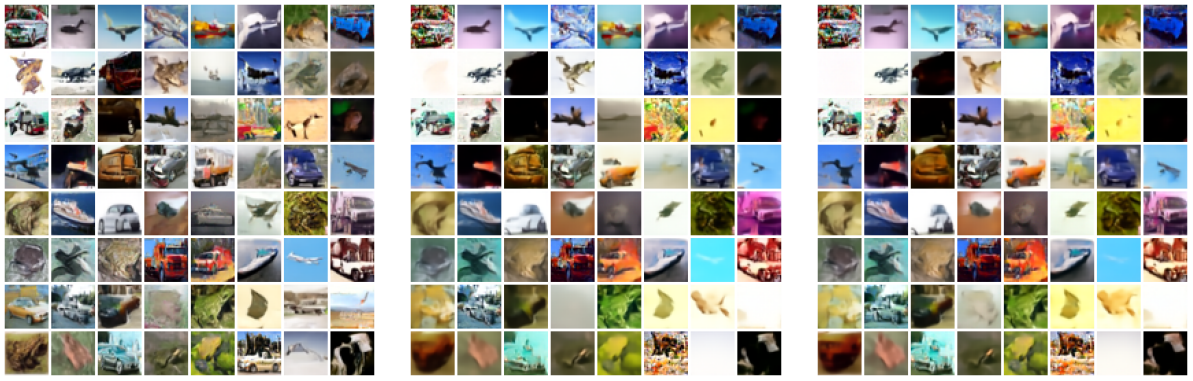


Figure 3: Metrics across generations for the baseline and CDR-W (dashed: ideal or perfect generator).

- **The weights point the right way.** For M1 the ratios were 0.73 for airplane, 0.86 for frog, 8.4 for dog and 10 (the cap) for horse. The weighted training data had a common-group share of 93.1% (D1 itself: 94.8%).
- **Most of the composition drift is removed.** KL stays near 0.065 instead of rising to 0.284. At D3 airplane is at 27% instead of 41% and truck at 11.7% instead of 4.3%.
- **The tail is better kept, not restored.** Classifier TRR is 0.68 against 0.52. Deer (0.65% against 0.39%) and dog (0.27% against 0.08%) improve. Horse, with only 50 training images, does not (0.07% against 0.12%) even though its weight was at the cap: reweighting a few images cannot add information that is not there. Cat is over-produced in both chains.
- **Quality is not fixed.** FD is lower (994 against 1072), but recall and the unrecognised share are slightly worse. The weights redistribute the training signal between classes; they cannot stop the general loss of quality from training on synthetic data.

7.3 Qualitative results

Figure 4 shows 64 samples from each model with a fixed seed. M0's samples have clear object shapes. The second-generation samples of both chains are blurrier and lower in contrast, and more of them are flat colour fields or almost empty frames, which is what the rising unrecognised share measures. The baseline's samples are dominated by airplane-like shapes on sky backgrounds and frog-like textures; CDR-W's show more variety, but the same loss of detail.



M0 (D1) Baseline M2 (D3) CDR-W M2 (D3)

Figure 4: Samples from M0 and from the second-generation models of both chains.

7.4 Sampling-time refinement without training

After the run we tested three ways to improve the trained checkpoints without any training, on CPU with the measurement classifier (512 samples per model, seed 0, 25 DDIM steps).

- **Weight interpolation** $\theta = \frac{1}{2}(\theta_{M0} + \theta_{\text{CDR-W M2}})$ produces pure noise (54% unrecognised on 128 samples). The models were trained independently from scratch, so their parameters do not line up.
- **Classifier guidance** on the predicted clean image (scales 0.05 and 0.15) hit the requested class only 20–27% of the time and raised the unrecognised share.
- **Refinement with M0** (SDEdit-style [9]): an image from a second-generation model is forward-noised to $t = 500$ and the DDIM trajectory is finished by M0. Both models keep their trained weights.

Sampler	High	Mid	Low	Unrecognised
M0	95.9%	3.2%	1.0%	19.9%
Baseline M2	96.4%	2.4%	1.2%	34.8%
CDR-W M2	94.8%	4.0%	1.2%	36.3%
Baseline M2 + M0 refinement	92.0%	6.0%	2.0%	21.9%
CDR-W M2 + M0 refinement	92.5%	6.3%	1.3%	22.3%

Table 7: Training-free refinement (512 samples per model; the low group is only a few images, so its share is noisy). Original D0 mix: 89.4% / 8.4% / 2.2%.

Refinement brings the unrecognised share back to about M0's level (22% against 35–36%), and the images are visibly sharper (Figure 5). It also moves the class mix towards the original, but it does so for the baseline as much as for CDR-W, so this shift comes from the refinement and not from CDR-W. Because M0 was trained on real data, the refinement re-introduces real-data knowledge at sampling time; it is a practical quality fix, not something the second-generation models learned.

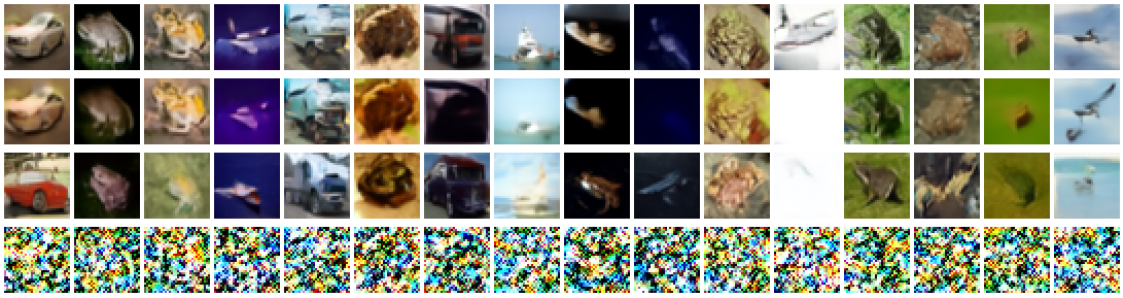


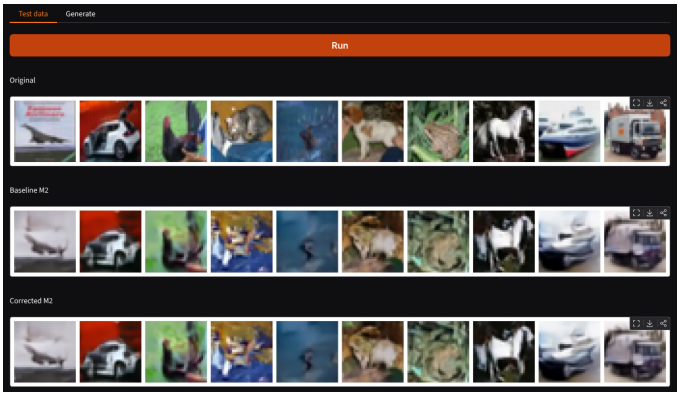
Figure 5: Same initial noise, rows from top: M0, CDR-W M2, CDR-W M2 + M0 refinement, weight interpolation of M0 and CDR-W M2.

8 Test Interface

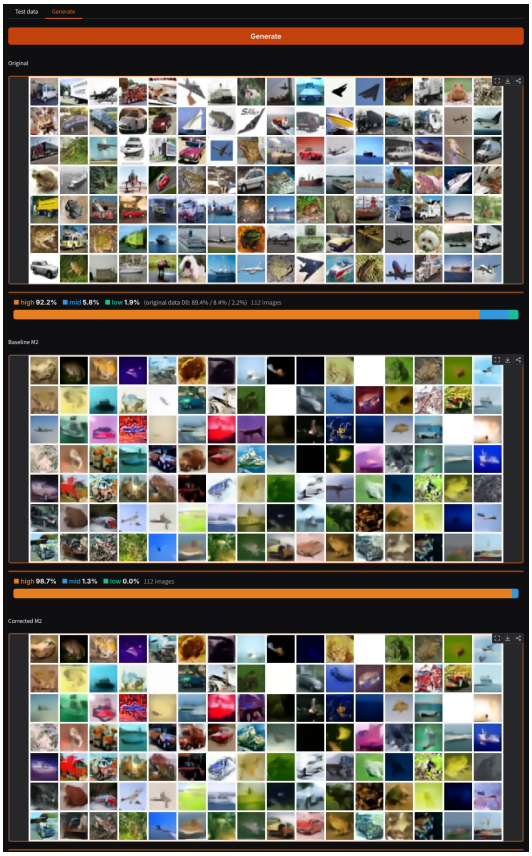
The interface (`demo_app.py`, Gradio) loads the trained checkpoints and the measurement classifier and runs them live. It is kept deliberately simple for the demo, with fixed settings so that every run takes the same time.

- **Test data.** Ten held-out CIFAR-10 test images (the first test image of each class, never used for training) are forward-noised to $t = 250$ and denoised with 20 DDIM steps by Baseline M2 and by the corrected model (CDR-W M2). Rows: original, Baseline M2, corrected M2. This shows the trained networks acting on real test data.
- **Generate.** The same initial noise is passed through both models and the images appear in batches of 16, up to 128 per row. After each batch the classifier measures the high/mid/low shares, and a bar under each row grows with them. The top row shows real test images drawn in the D0 class proportions as a reference.

Implementation: the UNets are rebuilt from the configuration stored in each checkpoint (a `UNet2DModel` with 24,318,083 parameters) and are sampled with the same deterministic DDIM used in the pipeline. The initial noise comes from a seeded CPU generator, so every model receives identical noise on any device (CPU, CUDA or Apple MPS). Shares are computed only over images the classifier recognises with confidence ≥ 0.8, as in the main evaluation. A constant in the file switches the corrected model to the refinement pipeline of Section 7.4.



(a) Test data



(b) Generate (after 112 images)

Figure 6: The test interface running on the trained checkpoints.

9 Plan for Novelty Assessment

- **Compensate the generator's own bias.** CDR-W fixes the composition of the training data, but the next model still adds its own bias while learning it (M0 already turns 89.4% into 94.8%). We have a variant (`cdr_w_comp`) that multiplies the ratio by $(\tilde{p}_0/\hat{p}_{D1})^{\lambda}$ to pre-compensate this; we will test it with $\lambda = 1$.
- **Sampling-time correction.** Combine CDR-W with minority guidance [4] to help the rarest classes, which loss weighting cannot recover, and replace M0 in the refinement step by a model that has not seen real data, to test how much of its gain is real-data knowledge.
- **More reliable evidence.** Repeat the run with more seeds for error bars, add a third generation, and report per-class recall to separate "not generated" from "generated but unrecognisable".

10 Conclusion

We built an imbalanced version of CIFAR-10 and a loop in which each unconditional diffusion model is trained from scratch on the previous model's output. The first model already over-produces the common classes, and two rounds of recursion increase the drift of the class mix by almost seven times (in KL) while image quality and diversity collapse. Our mitigation, class-density-ratio weighted

training, weights the diffusion loss by the ratio of original to generated class proportions, using the same images and compute as the baseline. It removes most of the extra drift and improves tail retention, but it does not restore the rarest class or stop the quality loss. A training-free refinement with M0 removes much of the quality loss at sampling time, though it relies on a model trained on real data. Our results come from one seed and two generations, so we treat them as a first, partial success rather than a complete fix.

References

[1] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data, Nature, 2024. https://www.nature.com/articles/s41586-024-07566-y

[2] Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoohi, and Richard G. Baraniuk. Self-Consuming Generative Models Go MAD, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2307.01850

[3] Sierra Wyllie, Ilia Shumailov, and Nicolas Papernot. Fairness Feedback Loops: Training on Synthetic Data Amplifies Bias, ACM Conference on Fairness, Accountability, and Transparency (FAccT), 2024. https://arxiv.org/abs/2403.07857

[4] Soobin Um, Suhyeon Lee, and Jong Chul Ye. Don't Play Favorites: Minority Guidance for Diffusion Models, International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2301.12334

[5] Yiming Qin, Huangjie Zheng, Jiangchao Yao, Mingyuan Zhou, and Ya Zhang. Class-Balancing Diffusion Models, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023. https://arxiv.org/abs/2305.00562

[6] Zachary C. Lipton, Yu-Xiang Wang, and Alexander J. Smola. Detecting and Correcting for Label Shift with Black Box Predictors, International Conference on Machine Learning (ICML), 2018. https://arxiv.org/abs/1802.03916

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising Diffusion Probabilistic Models, Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/abs/2006.11239

[8] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising Diffusion Implicit Models, International Conference on Learning Representations (ICLR), 2021. https://arxiv.org/abs/2010.02502

[9] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided Image Synthesis and Editing with Stochastic Differential Equations, International Conference on Learning Representations (ICLR), 2022. https://arxiv.org/abs/2108.01073

[10] Muhammad Ferjad Naeem, Seong Joon Oh, Youngjung Uh, Yunjey Choi, and Jaejun Yoo. Reliable Fidelity and Diversity Metrics for Generative Models, International Conference on Machine Learning (ICML), 2020. https://arxiv.org/abs/2002.09797

[11] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, et al. DINOv2: Learning Robust Visual Features without Supervision, Transactions on Machine Learning Research (TMLR), 2024. https://arxiv.org/abs/2304.07193

[12] Alex Krizhevsky. Learning Multiple Layers of Features from Tiny Images, Technical report, University of Toronto, 2009. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[13] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, Yiyi Xu, Steven Liu, and Thomas Wolf. Diffusers: State-of-the-art diffusion models, 2022, https://github.com/huggingface/diffusers. Accessed on: September 12, 2026.